# Smart Attendance System

## Final Technical Project Report

Prepared for final project review and delivery

### 1. Project Overview

This project is a Flask-based smart attendance management system for educational institutions. It supports teacher administration, student access, attendance marking, student profile management, face-based attendance, attendance reporting, export tools, remote sync support, and role-based security.

### 2. Main Features

• Teacher and admin login with secure password validation and lockout protection.

• Student portal login and attendance tracking visibility.

• Student management with create, edit, archive, CSV import/export, and enrollment records.

• Weekly timetable management and overlap validation.

• Manual and face-based attendance marking.

• Student photo validation to ensure real face images are used for recognition.

• Attendance dashboard and analytics reporting.

• CSV, XLSX, and PDF-style reporting and export workflows.

• Role-based access control, session timeout, and audit logging.

• Remote sync queue for attendance data transfer and ingestion.

• Notifications and alerts for low attendance or risk conditions.

• AI face recognition backend with OpenCV fallback for robust recognition.

### 3. Project Structure

• app.py — application entry point and startup.

• attendance/__init__.py — Flask app factory, session configuration, blueprint registration, default admin creation.

• attendance/auth.py — login, lockout handling, user management, audit export, role checks.

• attendance/students.py — student CRUD, photo upload, CSV import/export, student portal integration.

• attendance/attendance_routes.py — attendance dashboard, marking routes, dashboard logic, student alerts.

• attendance/reports.py — analytics and export reporting.

• attendance/face.py — face detection and matching with AI and OpenCV fallback.

• attendance/db.py — database initialization, schema setup, migrations and DB access.

• attendance/security.py — login_required and role_required wrappers.

• attendance/sync_routes.py — remote sync status and queue processing.

• schema.sql — SQLite schema used by the project.

• templates/ — all web UI templates for login, dashboard, attendance marking, reports, and student pages.

• static/ — CSS and JavaScript assets.

• uploads/students/ — student photo storage.

• tests/test_app.py — regression and functional verification suite.

## 4. Technology Stack

• Python 3.14 environment with Flask 3.x

• SQLite database for local persistence

• Jinja2 templates and Werkzeug security

• OpenCV and face_recognition for face attendance

• Pillow for image handling

• ReportLab for PDF generation

• qrcode and QR-based attendance support

• OpenPyXL for Excel exports

• pytest for automated regression tests

## 5. Role and Access Model

The system distinguishes between administrator/teacher roles and student roles. Access is protected with session enforcement, timeouts, and role-specific routes. Default credentials are created automatically for admin and teacher users to simplify setup.

## 6. Key Functional Fixes Applied During Final Hardening

• Made default admin and teacher user creation idempotent so repeated app startup does not crash.

• Enabled AI attendance mode by default when support is available.

• Corrected the face-recognition matching logic to tolerate realistic live-camera variation without false negatives.

• Validated student uploaded images to reject empty or non-face photos.

• Fixed student portal low-attendance alert generation so alerts are created automatically when thresholds are crossed.

• Added lockout protections for repeated failed logins and safe audit logging.

• Hardened session expiry handling to redirect stale sessions to login.

• Handled the legacy face_recognition_models pkg_resources deprecation warning by suppressing the exact upstream warning without hiding unrelated warnings.

• Ensured reporting routes and export logic remain stable under regression testing.

## 7. Security and Reliability Notes

The project includes secure password hashing, login attempt tracking, protection against duplicate student codes, validation for timetable overlaps, and session controls. It also maintains audit logs and sync queue entries to make data movement and access more traceable.

## 8. Setup Instructions

• Create and activate a virtual environment.

• Install dependencies using requirements.txt and requirements-ai.txt.

• Run the app with: python app.py

• Access the app through the browser on the configured local port.

• Default admin credentials: admin / admin123

• Default teacher credentials: teacher / teacher123

• Optional AI module support: ENABLE_AI_MODULE=true

## 9. Project Validation Result

The final verification was completed in the project environment using the command below:

source .venv-1/bin/activate && python -m pytest -q tests/test_app.py

Result: 21 passed, 0 failed in 3.96s.

## 10. Final Assessment

This project is in a working and production-ready state for the current codebase. The attendance workflow, login security, student portal, face recognition, reporting, and export features all pass the final regression suite. The remaining upstream warning comes from the third-party face_recognition_models package and was addressed in a controlled way without disrupting functionality.

End of report